SDN-Based Firewall

using Mininet and POX

Name : Dikshith Varma M N

Class : AIML-G

SRN : PES1UG24AM363

1. Introduction to SDN

Software Defined Networking (SDN) is a modern networking approach that separates the control plane from the data plane.

- The data plane (switches/routers) is responsible for forwarding packets.
- The control plane (controller) makes decisions about how traffic should flow.

In traditional networks, both these functions are tightly coupled within network devices, making it difficult to manage and configure networks dynamically. SDN solves this problem by introducing a centralized controller that can program network behavior.

Advantages of SDN

- Centralized control of the network
- Easier implementation of policies (like firewall rules)
- Better scalability and flexibility
- Dynamic traffic management

2. Objective of the Project

The objective of this project is to implement a **SDN Based Firewall** using SDN principles. The firewall should:

- Allow normal communication between selected hosts
- Block specific traffic based on predefined rules
- Demonstrate controller-based decision making
- Show flow rule installation in switches

3. Tools and Technologies Used

- Mininet – Network emulator used to create virtual topology
- POX Controller – SDN controller used to implement firewall logic
- Python – Programming language used for controller development
- OpenFlow Protocol – Used for communication between switch and controller
- iperf – Used for performance testing (throughput measurement)

4. Network Topology

The project uses a Single Switch Topology consisting of:

- 1 Switch (s1)
- 3 Hosts (h1, h2, h3)
- 1 Remote Controller (POX)

All hosts are connected to a central switch, which is controlled by the POX controller.

Topology Diagram

```
        h1
         |
         |
        s1
       /  \
    h2     h3
```

5. Working of the System

When a packet arrives at the switch, it is sent to the controller as a packet_in event. The controller then analyzes the packet — specifically the source IP and destination IP — and makes a forwarding decision based on predefined rules:

- If traffic matches a blocked rule → DROP
- Otherwise → FORWARD

The controller installs a flow rule in the switch using OpenFlow, ensuring subsequent matching packets are handled directly by the switch without re-consulting the controller.

6. Firewall Implementation

Rule Implemented

Block traffic from h1 (10.0.0.1) → h2 (10.0.0.2)

Logic

- Extract IP addresses from incoming packets

- Compare with blocked list
- If match → install drop rule with high priority
- Else → forward packet normally

This ensures that blocked traffic is stopped before reaching the destination.

7. Testing and Results

7.1 Ping Test

Test Case	Result
h1 → h2	Blocked (100% packet loss)
h2 → h3	Successful communication

Blocked Ping (h1 → h2)

```
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
^C
--- 10.0.0.2 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3052ms
```

Allowed Ping (h2 → h3)

```
mininet> h2 ping h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=96.0 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.100 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.089 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=0.067 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=0.126 ms
64 bytes from 10.0.0.3: icmp_seq=6 ttl=64 time=0.081 ms
64 bytes from 10.0.0.3: icmp_seq=7 ttl=64 time=0.059 ms
^C
--- 10.0.0.3 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6136ms
rtt min/avg/max/mdev = 0.059/13.794/96.040/33.576 ms
```

7.2 Performance Test using iperf

Test Case	Result
h3 → h2	High throughput (successful)
h1 → h2	Connection failed / blocked/ Blank Output

Iperf Allowed (h3 → h2)

```
mininet> h2 iperf -s &
mininet> h3 iperf -c h2
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.3 port 39738 connected with 10.0.0.2 port 5001 (icwnd/mss/irtt=1
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-10.0015 sec  55.4 GBytes  47.6 Gbits/sec
mininet>
```

iperf Blocked (h1 → h2)

```
mininet> h2 iperf -s &
mininet> h1 iperf -c h2
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
```

7.3 Flow Table Verification

Flow rules installed in the switch were verified using:

```
sudo ovs-ofctl dump-flows s1
```

Observed:

- Drop rule for blocked traffic
- Forwarding rules for allowed traffic

Flow Table Output

```
dikshith@legacy:~/pox$ sudo ovs-ofctl dump-flows s1
[sudo] password for dikshith:
 cookie=0x0, duration=215.244s, table=0, n_packets=15, n_bytes=1254, priority=200,ip,nw_src=10.0.0.1,nw_dst=10.0.0.2 actions=drop
 cookie=0x0, duration=224.155s, table=0, n_packets=0, n_bytes=0, priority=10,dl_src=7e:47:27:7a:79:1b,dl_dst=33:33:ff:7a:79:1b actions=FLOOD
 cookie=0x0, duration=224.115s, table=0, n_packets=0, n_bytes=0, priority=10,dl_src=f2:f1:1f:19:e0:bb,dl_dst=33:33:ff:19:e0:bb actions=FLOOD
 cookie=0x0, duration=224.073s, table=0, n_packets=2, n_bytes=180, priority=10,dl_src=ba:51:53:b1:11:a2,dl_dst=33:33:00:00:00:16 actions=FLOOD
 cookie=0x0, duration=224.010s, table=0, n_packets=2, n_bytes=180, priority=10,dl_src=f2:f1:1f:19:e0:bb,dl_dst=33:33:00:00:00:16 actions=FLOOD
 cookie=0x0, duration=223.503s, table=0, n_packets=2, n_bytes=180, priority=10,dl_src=7e:47:27:7a:79:1b,dl_dst=33:33:00:00:00:16 actions=FLOOD
 cookie=0x0, duration=223.186s, table=0, n_packets=5, n_bytes=350, priority=10,dl_src=ba:51:53:b1:11:a2,dl_dst=33:33:00:00:00:02 actions=FLOOD
 cookie=0x0, duration=223.122s, table=0, n_packets=5, n_bytes=350, priority=10,dl_src=f2:f1:1f:19:e0:bb,dl_dst=33:33:00:00:00:02 actions=FLOOD
 cookie=0x0, duration=223.122s, table=0, n_packets=5, n_bytes=350, priority=10,dl_src=7e:47:27:7a:79:1b,dl_dst=33:33:00:00:00:02 actions=FLOOD
 cookie=0x0, duration=215.294s, table=0, n_packets=0, n_bytes=0, priority=10,dl_src=7e:47:27:7a:79:1b,dl_dst=ff:ff:ff:ff:ff:ff actions=FLOOD
 cookie=0x0, duration=215.290s, table=0, n_packets=2, n_bytes=84, priority=10,dl_src=ba:51:53:b1:11:a2,dl_dst=7e:47:27:7a:79:1b actions=output:"s1-eth1"
 cookie=0x0, duration=207.561s, table=0, n_packets=0, n_bytes=0, priority=10,dl_src=ba:51:53:b1:11:a2,dl_dst=ff:ff:ff:ff:ff:ff actions=FLOOD
 cookie=0x0, duration=207.557s, table=0, n_packets=1251160, n_bytes=59560446340, priority=10,dl_src=f2:f1:1f:19:e0:bb,dl_dst=ba:51:53:b1:11:a2 actions=output:"s1-eth2"
 cookie=0x0, duration=207.511s, table=0, n_packets=437982, n_bytes=28910560, priority=10,dl_src=ba:51:53:b1:11:a2,dl_dst=f2:f1:1f:19:e0:bb actions=output:"s1-eth3"
 cookie=0x0, duration=166.903s, table=0, n_packets=1, n_bytes=42, priority=10,dl_src=7e:47:27:7a:79:1b,dl_dst=ba:51:53:b1:11:a2 actions=output:"s1-eth2"
dikshith@legacy:~/pox$
```

7.4 Network Connectivity Test (pingall)

The pingall command was used to verify overall connectivity between all hosts in the network.

```
mininet> pingall
```

Observation:

- Some packets were dropped due to firewall rules
- Communication between allowed hosts was successful
- Blocked hosts were unable to communicate

pingall Output

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
X h3
h2 -> h3
X h3
h3 -> h1 h2
*** Results: 33% dropped (4/6 received)
```

7.5 Firewall Execution (Controller Output)

The firewall module was executed in the POX controller using the following command:

```
python3.8 pox.py firewall
```

Observation:

- The controller started successfully
- Firewall module was loaded
- Logs displayed blocked traffic events

Example log output:

```
BLOCKED: 10.0.0.1 → 10.0.0.2
```

Firewall Command Execution

```
dikshith@legacy:~/pox$ nano  firewall.py
dikshith@legacy:~/pox$ python3.8 pox.py log.level --INFO firewall
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:firewall:🔥 Final Firewall + Learning Switch Loaded
WARNING:version:Support for Python 3 is experimental.
INFO:core:POX 0.7.0 (gar) is up.
```

Firewall Logs (Blocked Traffic)

```
INFO:firewall:Switch [00-00-00-00-00-01 2] connected
INFO:firewall:🚫 BLOCKED: 10.0.0.1 → 10.0.0.2
```

8. Analysis

The results obtained from the experiments clearly demonstrate the effectiveness of the implemented SDN-based firewall:

- Traffic from h1 to h2 resulted in **complete packet loss**, confirming that the firewall rule was successfully enforced.
- Communication between allowed hosts (h2 and h3) showed **normal latency and stable throughput**, indicating that legitimate traffic was unaffected.
- Performance testing using iperf showed **high bandwidth for allowed flows** and **no connection for blocked flows**, validating both functional and performance aspects.
- Flow table inspection using ovs-ofctl dump-flows confirmed that the controller correctly installed **high-priority drop rules** for blocked traffic and forwarding rules for permitted communication.
- Overall, the system demonstrates how SDN enables **fine-grained, centralized control** over network behavior.

9. Conclusion

This project successfully demonstrates the design and implementation of an **SDN-based firewall** using Mininet and the POX controller. By leveraging the separation of control and data planes, the controller was able to dynamically manage network traffic using programmable flow rules.

The firewall effectively blocks unauthorized communication (h1 → h2) while allowing legitimate traffic (h2 → h3), ensuring both **security and efficiency**. The use of tools like ping, iperf, and flow table inspection provided clear validation of the system's functionality.

Overall, this project highlights the advantages of SDN in enabling **flexible, scalable, and easily configurable network security mechanisms**, making it a powerful approach for modern network management.